HERMES · PHASE 1 · LESSON 0009 · LAB EXERCISE 07-TOOL-LOOP · ~45–60 MIN

Bounds and Termination

Lesson 0008's loop stopped at a hard-coded 4. Who owns the bounds, and what stops a reply that is already being generated?

> **Why this matters for Hermes**
> Scenario step S6 (the budget is enforced) is the cost guarantee. The job declares a call cap, a token ceiling and a deadline, and the supervisor enforces all three. Responsibility ⑥, cancellation, stops being a cost lever and becomes the enforcement mechanism. You predicted this design after lesson 0004: an in-flight stop must act on an estimate, because the true count arrives last. Today the lab measures your prediction.

1 · The bounds move into the spec

A literal in code is a bound nobody declared. Lesson 0008's `MAX_MODEL_CALLS` kept the loop correct, but no operator chose it and no spec file could change it. It is gone. The schema from lesson 0007 now carries every bound the loop obeys.

```
maxTokens:         z.number().int().positive().max(4096).default(1024),
costCeilingTokens: z.number().int().positive(),
maxModelCalls:     z.number().int().positive().max(20).default(4),
deadlineMs:        z.number().int().positive().default(60_000),
```

Four limits, and they divide cleanly. The provider enforces `maxTokens`, per call: a reply that reaches it stops with the wire's `max_tokens` stop reason. Hermes enforces the other three, across the whole job. The two new fields default rather than demand, because a spec that says nothing must still terminate. Here is Part E's spec file, and what an admitted spec becomes:

```
// specs/tight-budget.json — 6 keys in     // the admitted TaskSpec — 8 keys out
{ "title": "Audit the atlas graph under a tight budget",
  "owner": "themis",
  "instruction": "Summarize the atlas…",
  "maxTokens": 128,
  "costCeilingTokens": 190,               // maxModelCalls: 4   (default)
  "allowedTools": ["graph_health"],       // deadlineMs: 60000  (default)
  "outputPath": "vault/audits/…" }
```

The report changed with the spec. *Termination* is the guarantee that a job ends after finitely many model calls, with a classified outcome. The outcome union gains two arms, so a bound that fires is named, not buried in a note:

```
outcome: "landed" | "retry_later" | "gave_up" | "over_budget" | "out_of_time"
```

Layer	Changed?	Evidence
the six responsibilities, ① endpoint to ⑥ cancellation	no	the SDK still runs all six, below the port
the request body	yes	`"stream": true` joins every call the adapter builds
the delivery	yes	lesson 0004's event grammar, on every call
the assembled `Message`	no	byte-equal to the JSON path, measured in lesson 0004
the port	yes	an `onProgress` option, and partial text on the aborted arm
who enforces the bounds	yes	Hermes. The provider only ever enforced `maxTokens`

> **New today.**
> Five ideas are new.
>
> - Every bound comes from the spec, and every exit is a classified outcome.
> - Enforcement runs at three moments: before a call, during it, and after it.
> - The true cost arrives too late to enforce, so the in-flight check acts on an estimate.
> - One `AbortController` serves every bound, so cancellation becomes enforcement.
> - An aborted generation still lands, because the partial text is kept.
>
> Reprise: the event grammar and the abort from lesson 0004, the boundary parse from lesson 0005, the spec from lesson 0007, and the loop from lesson 0008.

2 · Enforcement has three moments

The gateway streams every call now. The reason is a single sentence: a bound that is only checked after a reply lands cannot stop the reply. Streaming gives the supervisor a view into a running call, and the port carries that view as a third discriminated union, narrowed on `kind`:

```
export type CallProgress =
  | { kind: "call_started"; inputTokens: number }   // e.g. { kind: "call_started", inputTokens: 110 }
  | { kind: "text"; chars: number };                // e.g. { kind: "text", chars: 22 }
```

Read what the union does not offer: output tokens. The wire explains why. Part E's second call opened with this frame, captured through the logging proxy:

```
event: message_start
data: {"type":"message_start","message":{…,"content":[],"stop_reason":null,
       "usage":{"input_tokens":110,"output_tokens":1,…}}}
```

The input count is true from the first frame, because the provider has read the request. The output count is a placeholder, because those tokens do not exist yet. The truth arrives in the last frame of a finished reply:

```
event: message_delta
data: {"type":"message_delta","delta":{"stop_reason":"tool_use",…},
       "usage":{"output_tokens":42}}
```

So the supervisor enforces at three moments. Before a call, it refuses to start one when the ledger has reached the ceiling. During a call, it adds booked spend, the call's true input, and an estimated output: one token per three characters seen. Text averages about four characters per token, so dividing by three errs high, and erring high stops a generation early rather than late. After a call, the true counts arrive and the ledger books them.

An aborted call never receives its `message_delta`, so its true cost never arrives. Part E booked 191 tokens against a ceiling of 190, and the report says the number is an estimate. The check before the next call is what makes the small overshoot safe: a job at its ceiling never starts another call.

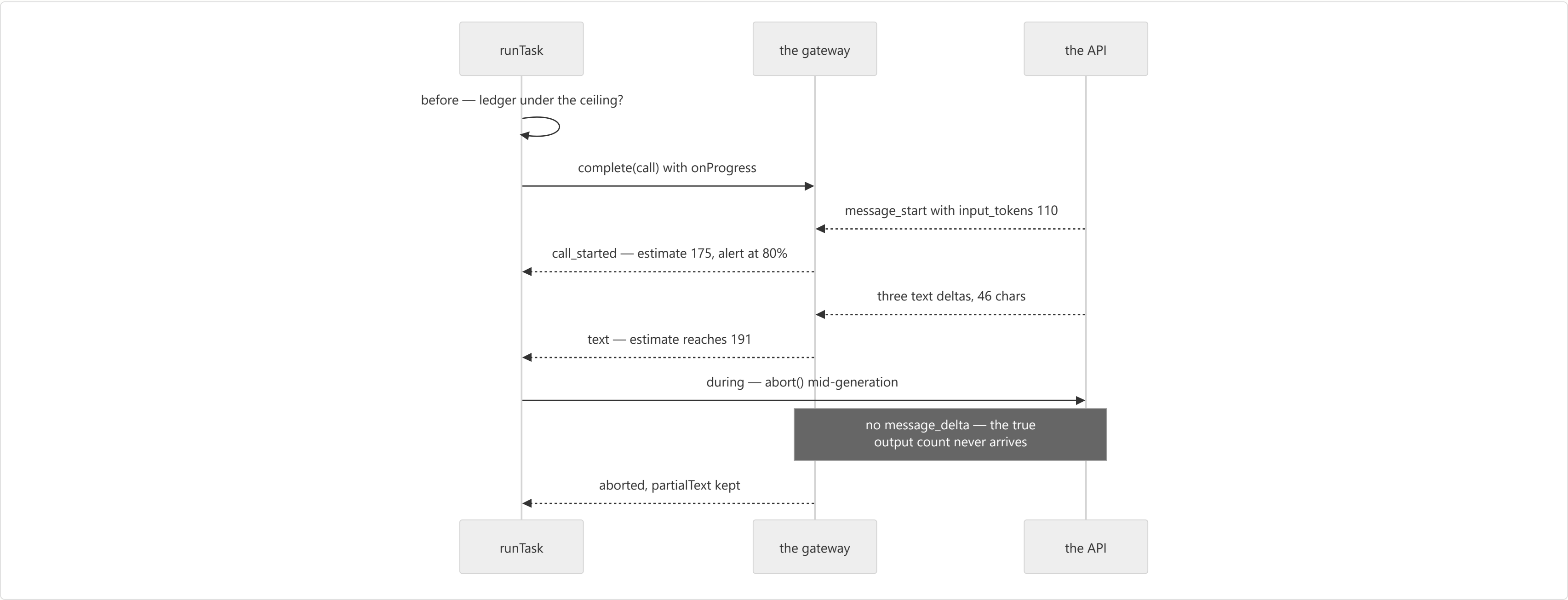
runTask
the gateway
the API
before — ledger under the ceiling?
complete(call) with onProgress
message_start with input_tokens 110
call_started — estimate 175, alert at 80%
three text deltas, 46 chars
text — estimate reaches 191
during — abort() mid-generation
no message_delta — the true output count never arrives
aborted, partialText kept
runTask
the gateway
the API

Diagram 1. Part E's second call. The truth arrives too late, so the abort acts on the estimate.

3 · One controller, every bound

Lesson 0004 taught the abort as a cost lever: the remainder of an aborted stream is never generated and never billed. Lesson 0009 pulls that trigger from policy. The supervisor arms one `AbortController` per job, and three hands can fire it:

```
const bounds = new AbortController();
const deadline = setTimeout(() ⇒ { boundHit = "deadline"; bounds.abort(); }, spec.deadlineMs);
const signal = options?.signal
  ? AbortSignal.any([options.signal, bounds.signal])
  : bounds.signal;
```

The static method `AbortSignal.any` takes an array of signals and returns one that aborts when any of them does. The operator's signal and the job's own bounds merge into one line to the gateway. The SDK tears the request down when that line fires, as measured in lesson 0004. The abort itself does not say why it happened. The supervisor knows, because it fired the controller, and `boundHit` records which bound pulled the trigger.

What the abort interrupts is not lost. A *partial artifact* is the output kept from a generation that was stopped before it finished. The gateway accumulates the streamed text and returns it on the aborted arm, and the report carries it toward S8 (outputs land). Part E kept 46 of 82 characters. Part F, with a 2400 ms deadline, ended at 2424 ms and kept 24. The remainder was never generated and never billed.

```
outcome: 'over_budget',   tokensSpent: 191,
notes: [ 'budget alert: estimated spend passed 80% of 190',
         'ceiling 190 reached mid-generation; spend is estimated — the true count never arrived',
         'partial artifact kept (46 chars): Audit complete. The tool reported: graph=atlas' ]
```

Part D closes the triangle offline. The fake answers instantly, so there is no mid-generation to watch, and the cap works between calls. A spec with `maxModelCalls: 2` ran against a fake scripted with three tool-hungry replies. The report says `gave_up` after two calls, and the third scripted reply was never requested.

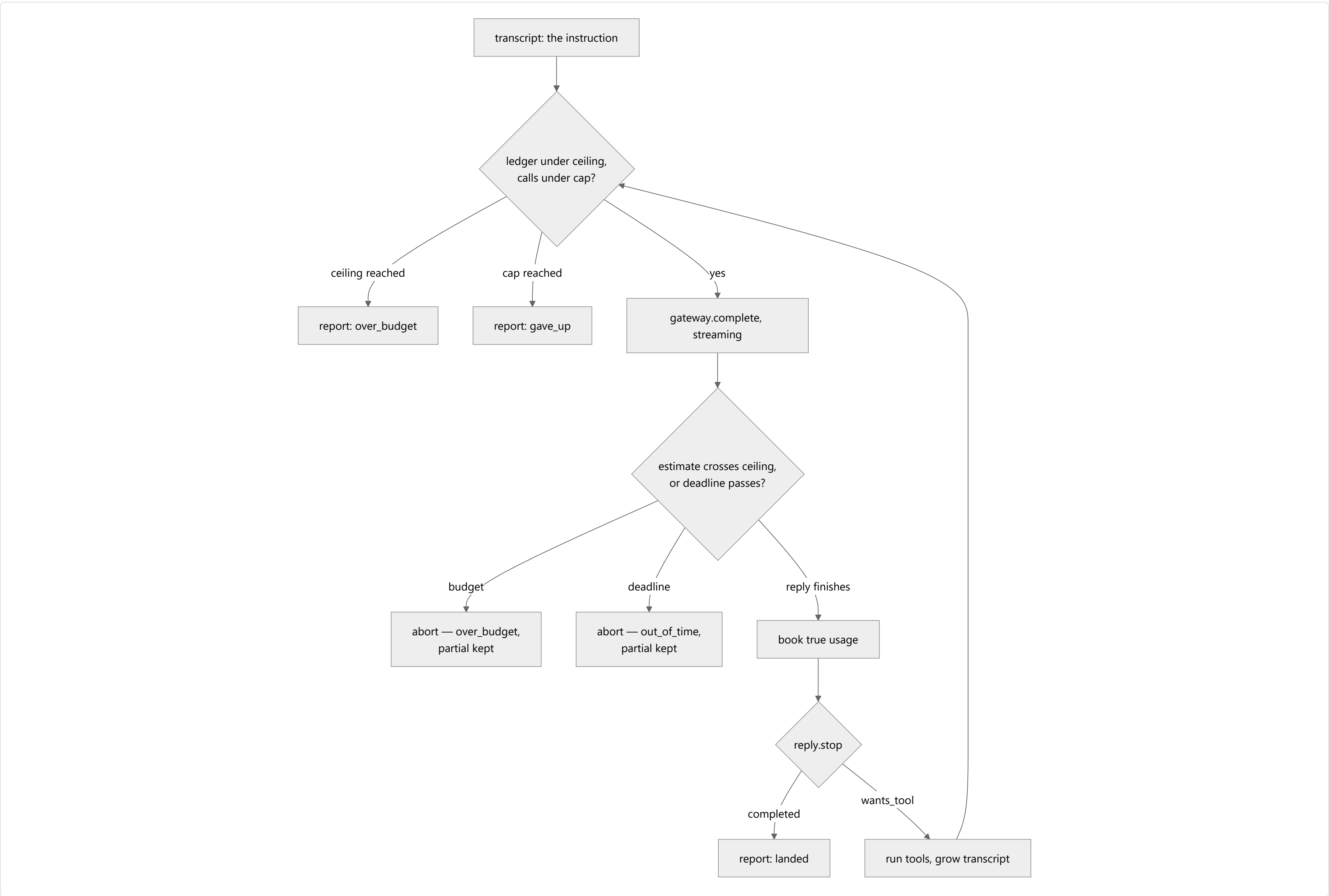
transcript: the instruction
ledger under ceiling, calls under cap?
ceiling reached
cap reached
yes
report: over_budget
report: gave_up
gateway.complete, streaming
estimate crosses ceiling, or deadline passes?
budget
deadline
reply finishes
abort — over_budget, partial kept
abort — out_of_time, partial kept
book true usage
reply.stop
completed
wants_tool
report: landed
run tools, grow transcript

Diagram 2. Every exit is a classified outcome. Failure arms from lesson 0008 are unchanged.

Capability	Status	Where it lives, or when it arrives
Call cap, budget and deadline from the spec	implemented	`task-spec.ts`, enforced in `supervisor.ts`, measured in Parts D, E and F
Mid-generation abort with partial kept	implemented	the gateway's aborted arm carries the streamed text
The partial artifact lands at `outputPath`	prepared	it lands in the report's notes. Nothing writes files yet
The 80% alert reaches an operator	prepared	it is a report note. No operator channel exists
An operator approves a tool call	planned	lesson 0010
The abort appears in a durable trace	planned	lesson 0011

4 · The exercise

Commands and the measured table live in `07-tool-loop/README.md`, steps 7 to 10. The arc:

1. Run Part D. Set `maxModelCalls` to 1, predict the report, then run again.
2. Run Part E with the mock's terminal visible. Match its last delta against the partial artifact in the report.
3. Raise the tight budget's ceiling to 250. Predict which bound fires now, then run.
4. Lower the deadline to 300 ms, and say where the abort lands before you run.

5 · Exercise: which moment enforces it?

Place each check at the moment it runs

Answer from memory before you click. Feedback is immediate.

Refusing to start a call because the ledger reached the ceiling.

before a call | during a call | after a call | the provider, per call

Aborting a generation because the estimate crossed the ceiling.

before a call | during a call | after a call | the provider, per call

Booking a call's true token counts into the ledger.

before a call | during a call | after a call | the provider, per call

Stopping one reply at the spec's `maxTokens`.

before a call | during a call | after a call | the provider, per call

Ending the job because the call cap is spent.

before a call | during a call | after a call | the provider, per call

Raising the alert at 80% of the ceiling.

before a call | during a call | after a call | the provider, per call

Score: 0 / 0

6 · Quiz

Four questions

Q1. Why does the gateway stream every call since this lesson?

- Streaming is cheaper, because deltas bill at a lower rate than one body
- The mock can only speak the tool grammar over its streaming path
- Enforcement needs to see inside a call, or it can only act between calls
- The SDK retries streamed requests, and retries make the loop terminate

Q2. Part E reports 191 tokens spent against a ceiling of 190. Why is the number an estimate?

- The mock rounds its token counts up, so every report overshoots by one
- The aborted call's final frame never arrived, so its true output count does not exist
- The ledger adds the tools array on every call, and its size is unknown
- The abort raced the last delta, so one chunk was counted two times

Q3. The gateway returns `kind: "aborted"`. How does the supervisor pick between `over_budget`, `out_of_time` and `gave_up`?

- It remembers which bound fired the controller, because the abort itself does not say
- It reads the reason from the failure, because the adapter classifies the cause
- It compares the ledger to the ceiling, and the larger number names the outcome
- It asks the provider, whose error body names the bound that was breached

Q4. Why can the provider enforce `maxTokens` but never `costCeilingTokens`?

- The ceiling is measured in dollars, and the provider only counts tokens
- The provider enforces nothing, and both bounds belong to the supervisor
- The ceiling arrives in the spec file, which the provider refuses to parse
- The API is stateless per call, and the ceiling spans calls only Hermes can see

Score: 0 / 0

7 · Say it in your own words

Answer these in the chat after the lab, with no notes.

1. Explain why the in-flight budget check acts on an estimate, and which way the estimate must err.
2. A reviewer proposes checking the budget only between calls, to avoid estimates entirely. Say what that design loses, in two sentences.
3. Name the three bounds a spec declares for the loop, and the outcome each one produces when it fires.

If you keep only four sentences from this lesson, keep these. Every bound a job obeys comes from its spec: a call cap, a token ceiling, and a deadline. Enforcement runs before a call, during it, and after it, and every exit is a classified outcome. The wire reports input tokens first and output tokens last, so the in-flight check acts on an estimate that errs high. One `AbortController` serves the budget, the deadline, and the operator, and an aborted generation still yields a partial artifact.

Primary source

Read the Anthropic streaming documentation and find where `usage` appears in `message_start` and in `message_delta`. Then compare its abort guidance with `supervisor.ts`, and note that the provider offers no budget of this kind: the ceiling is yours to enforce.

Version pin. Every excerpt and number here was verified against `@anthropic-ai/sdk` 0.113.0 and `zod` 4.4.3, by live runs against the exercise 01 mock. The wire frames came from a logging proxy in front of it. The mock's token counts are estimates and its canned output is 42 tokens. The measured ledgers are exact for this mock and illustrative for a real provider. The SSE grammar and block shapes are faithful to the API. The two new terms have notes under `wiki/terms/`.

Next. Lesson 0010, *Approval Gates and Permissions*, in the same lab. The loop can stop itself now. Next it learns to wait: which tool calls run, and which queue for a human.